

第 31 章 Podman 镜像、容器与 Rootless 运行

从镜像引用到当前实例：把用户上下文、端口、挂载、数据与运行证据放进同一条验证链。

对象模型操作语义验证诊断经典任务

大字号阅读版

本章阅读导航 先抓住一条主线：Podman 命令不是在修改一个笼统的“容器”，而是在不同用户上下文中依次处理镜像引用、本地镜像、容器配置对象、运行进程、宿主端口和外部数据。

01确认用户上下文rootless 状态属于目标宿主用户

02证明镜像身份完整引用、tag、digest 与本地 image ID

03建立容器配置名称、命令、环境、端口和挂载

04判断运行状态主进程、退出码、日志与 exec

05验收宿主链路port、ss、curl 与应用结果

06证明数据边界可写层、bind mount、volume 与删除结果

专题地图

知识专题	Registry、镜像引用、Tag 与 Digest
操作专题	拉取、列出、标记并检查镜像
知识专题	镜像、容器配置、可写层与运行进程
操作专题	创建、运行、启停与删除容器
知识专题	Rootless、用户命名空间与权限边界
操作专题	环境、端口、命令与用户身份
知识专题	可写层、Bind Mount 与 Named Volume
操作专题	SELinux :Z/:z 与挂载验证
操作专题	ps、inspect、logs、exec 与 port
诊断专题	按对象层定位退出、不可访问和权限拒绝
经典任务	部署并重建 Rootless Web 容器
经典任务	修复端口与 SELinux 挂载双故障

阅读时持续回答

1. 当前命令以哪个宿主用户执行？
2. 操作对象是镜像、容器配置还是运行进程？
3. running 只证明了哪一层？
4. 宿主端口和容器端口的方向是什么？
5. 数据位于可写层、宿主目录还是 named volume？
6. DAC、UID/GID 映射和 SELinux 哪一层拒绝了访问？
7. 删除容器时哪些数据会保留？
8. 当前证据是否越界声称了注销或重启后的持久运行？

第 31 章 · 正文

把一个应用“放进容器”以后，系统中并没有只出现一个新对象。Registry 中有镜像引用，目标用户的本地存储中有镜像；`podman create` 建立容器配置对象，`podman start` 才产生运行进程。端口发布把宿主 socket 与容器网络连接起来，挂载把宿主目录或 Podman volume 暴露给容器，而 rootless 运行又把这些状态限定在某个普通用户的命名空间、存储和权限范围内。最常见的误判都来自把这些对象合并成一句“容器已经好了”：镜像已拉取不等于实例已创建；容器`running` 不等于应用监听正确；`podman port` 有映射不等于宿主请求成功；容器内 UID 0 不等于宿主 root；删除容器后，bind mount 中的数据通常仍在，而只写入可写层的数据会失去唯一入口。本章沿着“用户上下文 → 镜像 → 容器配置 → 运行进程 → 端口与挂载 → 功能与数据”推进。用户级 systemd、linger 与 Quadlet 只在章末建立交接，完整内容归下一章《容器持久化、用户 systemd 与 Quadlet》；firewalld 和 SELinux 通用规则也不在本章重复展开。

概念

镜像引用（image reference）是定位 Registry 内容的名称体系，通常由 Registry、namespace/repository 和 tag 组成，也可用 digest 固定到具体内容。Tag 便于发布和阅读，但可以被重新指向；digest 更适合作为内容身份。操作时应按题目保留完整引用，并用 image ID、RepoDigests 和 inspect 字段证明实际拉取内容。

概念

镜像（image）是当前 Podman 存储中的只读模板和层集合，它可以拥有多个 tag，也能被多个容器共同引用。镜像存在只证明创建实例所需的模板已在当前用户存储中，并不能证明容器已经创建、进程正在运行或应用能够访问。

概念

容器实例（container instance）是从镜像创建的独立对象，保存名称、命令、环境、端口、挂载和可写层；启动后才出现容器主进程。停止只改变运行状态，删除则移除配置对象和可写层。理解这一层次，是区分 `create`、`run`、`start`、`stop` 和 `rm` 的基础。

概念

Rootless 与 Rootful 描述 Podman 由普通宿主用户还是宿主 root 运行。两者拥有不同的存储、容器、卷和认证状态；root 查询不到普通用户的容器，并不能证明对象不存在。考试要求某用户运行 rootless 容器时，login、pull、run、inspect 和删除都必须保持在该用户上下文。

概念

用户命名空间（user namespace）把容器内 UID/GID 映射到宿主身份范围。Rootless 容器内 UID 0 可以在命名空间内拥有管理语义，但不会因此变成宿主 UID 0，也不能绕过宿主目录的 DAC 权限或 SELinux。遇到挂载拒绝时，要同时调查调用用户、映射后的进程身份和文件标签。

概念

端口发布（port publishing）是创建配置中的宿主端口到容器端口映射，例如 `-p 8088:8080`。它只建立转发入口，不保证容器进程正在监听，也不保证 HTTP 请求成功。可靠验收必须继续检查容器状态、`podman port`、宿主 socket 和协议响应。

概念

Bind mount 与 Named volume 都能把数据放到容器可写层之外，但所有权不同。Bind mount 明确引用管理员选择的宿主路径；named volume 由 Podman 管理名称和实际目录。删除容器通常不会删除这两类外部数据，而容器可写层会随容器对象消失，因此删除前必须先用 inspect 证明数据来源。

概念

SELinux `Z` 与 `z` 是 Podman 挂载语法中的重标记请求。`Z` 适合当前容器私有使用，`z` 适合多个容器共享同一内容；两者都会修改宿主对象的标签，所以不能机械附加到系统共享目录。它们也不能替代正确的宿主路径、所有权、模式和 UID/GID 映射。

操作语义速查 在进入详细专题前，先把本章最关键的接口分成六组。速查区只建立命令的作用对象和参数方向；正文再解释证据边界、诊断分支和安全限制。

`podman pull` / `podman images` / `podman image inspect` **SYNOPSIS**

```
podman pull [OPTIONS] IMAGE
podman images [OPTIONS]
podman image inspect [OPTIONS] IMAGE [IMAGE...]
```

在当前用户的 Podman 存储中获取、列出并检查镜像。三条命令分别回答“拉取什么”“当前有哪些引用”“实际内容和默认配置是什么”。**重要参数 / 形式**
`podman pull REGISTRY/NS/IMAGE:TAG`：优先使用题目给定的完整引用，避免短名称歧义。
`podman images --format FORMAT`：选择 repository、tag、ID 等稳定字段建立清单。
`podman image inspect IMAGE --format FORMAT`：精确提取 ID、架构、默认用户、命令或 digest；字段需按目标版本核对。

`podman run` / `create` / `start` / `stop` / `rm` **SYNOPSIS**

```
podman run [OPTIONS] IMAGE [COMMAND [ARG...]]
podman create [OPTIONS] IMAGE [COMMAND [ARG...]]
podman start [OPTIONS] CONTAINER
podman stop [OPTIONS] CONTAINER
podman rm [OPTIONS] CONTAINER
```

创建参数位于镜像名前，容器内命令位于镜像名后。`run` 是创建并启动；`start` 只能启动已有对象，不能补改创建时的端口、环境和挂载。 **重要参数 / 形式** `--name NAME`：为实例建立稳定管理名称；名称冲突时先检查旧对象是否可复用。`-d`：后台运行，不保证主进程不会立即退出。`--rm`：容器退出后自动删除对象；会减少排错现场，不适合需要保留退出证据的任务。`--user UID[:GID]`：指定容器内进程身份；不等于切换宿主用户，也不能绕过 DAC 或 SELinux。

`podman ps` / `inspect` / `logs` / `exec` / `port` **SYNOPSIS**

```
podman ps [OPTIONS]
podman container inspect [OPTIONS] CONTAINER
podman logs [OPTIONS] CONTAINER
podman exec [OPTIONS] CONTAINER COMMAND [ARG...]
podman port CONTAINER [PRIVATE_PORT[/PROTO]]
```

这一组负责保存运行证据。`ps -a` 选择对象，inspect 读取创建配置和状态，logs 读取 stdout/stderr，exec 做定向容器内取证，port 显示发布映射。 **重要参数 / 形式** `podman ps -a`：同时显示 created、running、exited 等对象。`--format FORMAT`：提取明确字段；不要把整段 JSON 当作唯一可读证据。`podman logs --tail N NAME`：查看最近日志；日志为空不自动等于应用无错误。`podman exec NAME COMMAND`：只适用于 running 容器，并要求镜像中存在目标命令。

`podman volume` **SYNOPSIS**

```
podman volume create [OPTIONS] [NAME]
podman volume ls [OPTIONS]
podman volume inspect [OPTIONS] VOLUME [VOLUME...]
podman volume rm [OPTIONS] VOLUME [VOLUME...]
```

管理由 Podman 命名和定位的外部数据对象。Named volume 与容器对象生命周期分离，删除容器通常不会删除它。 **重要参数 / 形式** `podman volume create NAME`：创建稳定命名卷，便于重建容器时复用。`podman volume inspect NAME`：读取实际 Mountpoint 和元数据，证明容器配置引用的对象。`podman rm --volumes NAME`：可能同时处理关联的匿名卷；不要把它当作普通删除默认形式。

Rootless 身份与用户状态 **SYNOPSIS**

```
id
podman info
podman unshare COMMAND [ARG...]
loginctl show-user USER [OPTIONS]
```

确认当前宿主用户、rootless 存储上下文和 UID/GID 映射。`loginctl` 在本章只用于观察用户状态，不执行 linger 配置。 **重要参数 / 形式** `id`：证明当前宿主 UID、主组和附加组。`podman info`：确认 rootless、存储和运行时信息；输出字段随版本变化。`podman unshare cat /proc/self/uid\_map`：在 rootless 用户命名空间中观察映射，不伪造固定数值。`loginctl show-user USER -p State -p Linger`：只读取用户 manager 状态；持久运行配置归下一章。

端口与挂载验收 **SYNOPSIS**

```
ss -lnt
curl -fsS URL
ls -ldZ HOST_PATH
podman exec CONTAINER COMMAND [ARG...]
```

把 Podman 内部配置继续推进到宿主监听、协议功能和宿主数据。最终验收不能停在容器状态或命令返回码。 **重要参数 / 形式** `-p HOST\_PORT:CONTAINER\_PORT`：宿主端口在前，容器端口在后。`-e KEY=VALUE`：把环境变量固化到容器创建配置。`-v HOST:CONTAINER:Z`：私有 bind mount 重标记；使用前确认路径和共享关系。`-v VOLUME:CONTAINER`：使用 named volume；通过 volume inspect 和容器 inspect 联合验证。

[知识专题] 从 Registry 引用到本地镜像：名称、Tag 与 Digest

处理镜像的第一步不是直接输入 `podman pull nginx`，而是识别题目真正给出的镜像身份。短名称依赖本机 Registry 配置和别名；tag 可能被重新指向；同一个镜像 ID 也可以拥有多个名称。因此，稳定的路径是：先保存题目给出的完整引用，再通过本地镜像列表和 inspect 证明实际内容。

① [知识点] 完整镜像引用由多个可独立变化的部分组成

典型引用：

```
registry.lab.example.com/training/webapp:1.0
```

可拆成：

部分	示例	作用
Registry	registry.lab.example.com	定位提供镜像的服务
Namespace/Repository	training/webapp	组织镜像名称和访问范围
Tag	1.0	指向某个版本或发布通道的可读引用
Digest	sha256:...	对镜像内容的不可变标识

题目给出完整引用时，应优先原样使用。省略 Registry 的短名称需要由 registries.conf、短名称别名和交互选择来补全，批量或考试环境中容易产生歧义。

② [知识点] Tag 是名称，Digest 才是内容身份

Tag 可以移动。Registry 管理员可以让 webapp:stable 今天指向一个 digest，明天指向另一个 digest；本地系统已经拉取的旧镜像不会因为远端 tag 移动而自动变化。Digest 由镜像内容决定，更适合回答“实际拿到的是哪一份内容”。

这并不表示考试中必须把所有 tag 改成 digest。正确做法是：按题目指定的 tag 操作，同时用 podman image inspect 记录 RepoDigests、ID、架构和默认命令等证据。

③ [知识点] 一个镜像可以有多个名称，名称变化不等于复制内容

podman tag SOURCE TARGET 为本地镜像增加另一个引用。它通常不会复制所有镜像层。以下两个名称可以指向同一个 IMAGE ID：

```
localhost/webapp:latest
registry.lab.example.com/training/webapp:1.0
```

验证 tag 操作不能只看命令是否返回成功，应在 podman images 中核对新引用，并用 image ID 或 inspect 证明新旧名称指向预期内容。

④ [知识点] 本地镜像属于当前 Podman 存储上下文

Rootful Podman 默认使用系统级存储；rootless Podman 默认使用当前用户的数据目录。普通用户 websvc 拉取的镜像，不应假定 root 或另一个普通用户可以直接在自己的 podman images 中看到。

因此，题目要求“面向某用户运行 rootless 容器”时，镜像拉取、容器创建、状态查询和删除都应保持在同一目标用户上下文。

⑤ [知识点] Registry、运行默认值与存储配置来自不同文件

常见配置入口：

配置	系统级入口	用户级入口	本章用途
Registry 与短名称	/etc/containers/registries.conf	~/.config/containers/registries.conf	决定未限定名称如何补全、Registry 是否被标记为 insecure 或 blocked
Podman 默认值	/usr/share/containers/containers.conf、/etc/containers/containers.conf	~/.config/containers/containers.conf	影响部分命令默认行为
容器存储	/etc/containers/storage.conf	~/.config/containers/storage.conf	决定 graphroot、驱动等存储状态

命令行参数通常用于覆盖一次操作的默认值，但不能把所有配置文件都理解为同一种“整体覆盖”。遇到版本和加载顺序问题，应查当前系统的 podman(1)、containers.conf(5)、containers-registries.conf(5) 和 containers-storage.conf(5)。

[Cheatsheet] 完整引用优先；tag 可移动，digest 标识内容；podman tag 增加名称而不是复制层；镜像属于当前用户的 Podman 存储；Registry、运行默认值和存储分别查对应配置。

[操作专题] 拉取、列出、标记并检查镜像

镜像操作应形成一个小闭环：明确引用，拉取到目标用户存储，列出本地镜像，inspect 关键字段，需要时增加 tag，最后再次核对。podman search 不能作为“镜像一定存在”的可靠证明；真正的存在性和可用性由 Registry 请求、pull 结果和本地 inspect 共同决定。

① [操作] 以目标用户拉取完整镜像引用

作用对象：当前用户的本地镜像存储。

基本形式：

```
podman pull registry.lab.example.com/training/webapp:1.0
```

私有 Registry 需要认证时，先在同一用户上下文中执行：

```
podman login registry.lab.example.com
```

认证信息属于该用户。不要为了“让普通用户看到镜像”而改用 root 登录并拉取，因为 root 与普通用户的 Podman 状态是分离的。

验证：

```
podman images
podman image inspect registry.lab.example.com/training/webapp:1.0
```

② [操作] 用明确字段核对镜像身份

`podman images` 适合快速查看 repository、tag、image ID、创建时间和大小；`podman image inspect` 提供完整结构化配置。需要稳定提取时，可在目标版本支持的前提下使用 `--format`：

```
podman image inspect registry.lab.example.com/training/webapp:1.0 \
--format '{{.Id}} {{.Architecture}} {{.Os}}'
```

还应检查：

- RepoTags 与 RepoDigests；
- 镜像默认 `Cmd` / `Entrypoint`；
- 镜像默认 `User`；
- 暴露端口仅是镜像元数据，不等于宿主已发布端口。

③ [操作] 为导入或本地镜像增加目标 tag

```
podman tag localhost/webapp:latest \
registry.lab.example.com/training/webapp:1.0
```

验证：

```
podman images --format '{{.Repository}}:{{.Tag}} {{.Id}}'
```

如果题目还要求 push，那属于 Registry 发布操作；本章只建立 tag 身份和本地验证，不把构建与发布扩展为主线。

④ [操作] 从归档加载镜像时区分 `load` 与容器文件系统导入

考试材料可能提供由 `podman save` 生成的镜像归档。对应入口是：

```
podman load -i image.tar
```

或：

```
podman load < image.tar
```

`podman load` 恢复镜像归档及其名称信息；`podman import` 则把文件系统 tar 作为新镜像导入，语义不同。拿到文件前先判断它来自 image save 还是 container export，不要机械试命令。

⑤ [验证] 镜像闭环

```
目标用户身份
→ 完整引用
→ pull/load 成功
→ podman images 可见
→ image inspect 的 ID、digest、架构和默认配置符合目标
→ 需要的 tag 指向正确 image ID
```

[Cheatsheet] 同一用户执行 login/pull/images/inspect；短名不可靠时用完整引用；tag 后核对 image ID；镜像归档用 load，容器文件系统 tar 才考虑 import。

[知识专题] 镜像不是容器：配置对象、可写层与运行进程

容器生命周期至少包含三个层次：镜像提供只读初始内容；容器对象保存名称、命令、环境、端口和挂载配置，并拥有可写层；容器启动后才出现运行进程。podman ps 只显示其中一部分状态，因此排错时必须知道自己正在寻找哪一层。

① [知识点] Image 是模板，Container 是实例

同一镜像可以创建多个容器。它们共享镜像的只读层，但各自拥有：

- 容器 ID 和名称；
- 创建时的命令与参数；
- 环境变量；
- 端口发布；
- 挂载；
- 独立可写层；
- 启停与退出状态。

删除某个容器通常不会删除镜像，也不会影响从同一镜像创建的其他容器。

② [知识点] create、run 与 start 改变不同阶段

```
podman create
→ 建立容器对象，状态通常为 created

podman start
→ 启动已存在容器中的配置命令

podman run
→ create + start 的组合，并根据参数决定是否附着、后台运行或自动删除
```

podman start 不接受所有 podman run 参数，因为端口、挂载和环境变量已经在创建时固化。若这些配置错误，通常需要保留证据后删除并重新创建，而不是反复 stop/start。

③ [知识点] 容器的主进程决定生命周期

容器不是轻量虚拟机。容器的主进程退出，容器通常就进入 exited 状态。镜像默认命令如果是一次性命令，podman run -d 也不会让它永久运行。

因此，容器“立即消失”时第一条证据不是再次运行，而是：

```
podman ps -a
podman logs <CONTAINER>
podman container inspect <CONTAINER>
```

重点查看状态、退出码、实际命令和错误消息。

④ [知识点] Created、Running、Exited 与 Removed 是不同状态

状态	含义	可执行的典型下一步
created	对象存在，主进程尚未启动	podman start
running	主进程仍在运行	logs、exec、port、协议验证
exited	主进程已退出，对象仍在	logs、inspect、必要时 start

状态	含义	可执行的典型下一步
removed	容器对象已删除	只能重新 create/run

`podman ps` 默认只列出 running 容器；`podman ps -a` 才能看到 created、exited 等对象。

⑤ [知识点] 容器删除与数据删除必须分开判断

删除容器通常会移除容器配置和可写层，但以下内容不应混在一起：

数据位置	删除容器后的典型结果
容器可写层	随容器对象删除
bind mount 的宿主文件	仍由宿主管理，通常保留
显式命名的 named volume	通常保留，需单独 <code>podman volume rm</code>
匿名 volume	使用 <code>--rm</code> 或 <code>podman rm --volumes</code> 时可能被一并移除
本地镜像	不因普通 <code>podman rm</code> 被删除

删除前应先 `inspect` 挂载来源，避免把“容器可写层中的唯一数据”误认为持久数据。

[Cheatsheet] `create` 只建对象，`start` 启动旧对象，`run` 创建并启动；主进程退出即容器退出；`ps -a` 查非运行对象；删容器不等于删镜像、bind mount 或命名卷。

[操作专题] 创建、运行、停止、再次启动与删除容器

生命周期操作的关键不是把命令背成一串，而是每次操作前后都保存对象证据。名称冲突、错误端口、错误挂载和错误命令都可能迫使你重建容器；若先 `rm -f`，最重要的退出码、日志和配置就可能丢失。

① [操作] 只创建、不启动

```
podman create --name rh31-web \
    registry.lab.example.com/training/webapp:1.0
```

验证：

```
podman ps -a --filter name=rh31-web
podman container inspect rh31-web
```

适用场景：需要先核对创建配置，或后续由其他生命周期入口启动。一般考试任务更常直接使用 `podman run`。

② [操作] 创建并后台运行

```
podman run -d --name rh31-web \
    registry.lab.example.com/training/webapp:1.0
```

`-d` 只表示 Podman 不持续附着在前台，不保证应用不会退出。命令返回容器 ID 后，立即检查：

```
podman ps -a --filter name=rh31-web
podman logs rh31-web
```

③ [操作] 停止与再次启动

```
podman stop rh31-web
podman start rh31-web
```

`stop` 默认先请求主进程有序终止，并在超时后采取更强制措施。不要把 `podman kill` 当作常规停止入口。

验证：


```
podman ps -a --filter name=rh31-web
podman container inspect rh31-web \
  --format '{{.State.Status}} {{.State.ExitCode}}'
```

④ [操作] 删除前建立证据

推荐顺序：

```
podman ps -a --filter name=rh31-web
podman container inspect rh31-web > rh31-web.inspect.json
podman logs rh31-web > rh31-web.log 2>&1
podman stop rh31-web
podman rm rh31-web
```

在考试环境中不一定需要把证据写入文件，但思路应保持一致：先确认名称、状态、挂载和端口，再删除错误对象。

⑤ [诊断边界] 同名容器阻止 `run`

症状：

```
Error: the container name "rh31-web" is already in use
```

证据链：

```
podman ps -a --filter name=rh31-web
podman container inspect rh31-web
```

若旧对象配置正确，只是 exited，可直接 `start`；若端口、挂载或环境错误，保存证据后删除并按正确参数重建。不要看到名称冲突就直接 `podman rm -f`。

[Cheatsheet] `run -d` 后立刻 `ps -a`；stop/start 不改变创建参数；重建前保存 inspect 与 logs；名称冲突先判断旧对象是否可复用。

[知识专题] Rootless：用户命名空间、每用户状态与权限边界

Rootless 并不是“容器里不能有 root 用户”，而是容器进程由普通宿主用户发起，并通过用户命名空间映射 UID/GID。容器内 UID 0 可以在自己的命名空间中拥有一部分管理能力，但在宿主看来仍受普通用户权限限制。这是判断目录权限、端口和状态可见性的基础。

① [知识点] 容器内 UID 0 不等于宿主 UID 0

默认 rootless 映射中，调用 Podman 的宿主用户被映射为 rootless 用户命名空间的 UID 0。容器进程即使显示为 root，也不会自动获得读取宿主 `/root`、修改系统文件或绕过 SELinux 的能力。

确认宿主身份：

```
id
whoami
podman info
```

需要查看映射时，可使用：

```
podman unshare cat /proc/self/uid_map
podman unshare cat /proc/self/gid_map
```

这些输出依赖环境，不应编造固定数值。

② [知识点] `/etc/subuid` 与 `/etc/subgid` 提供附加映射范围

RHEL 创建普通用户时通常会为 rootless 容器配置 subordinate UID/GID 范围。静态检查入口：

```
grep '^websvc:' /etc/subuid /etc/subgid
```

如果映射缺失或被修改，实际处理可能需要管理员修复并执行与 Podman 版本匹配的迁移操作。本章只训练识别和证据，不展开所有高级 `--uidmap`、`--gidmap` 组合。

③ [知识点] 不同宿主用户拥有不同 Podman 状态

普通用户创建的容器不会自动出现在 root 或其他用户的 `podman ps -a` 中。以下两条命令查询的是不同状态集合：

```
sudo -iu websvc podman ps -a
sudo podman ps -a
```

因此，root 看到“没有容器”不能证明 websvc 没有 rootless 容器；root 拉取了镜像也不能证明 websvc 的本地镜像存储中已有该镜像。

④ [知识点] Rootless 对宿主目录权限提出双重要求

容器访问 bind mount 时，至少要同时满足：

1. 调用 Podman 的宿主用户能够遍历和访问源路径；
2. 映射后的容器进程 UID/GID 对宿主对象具有合适 DAC 权限；
3. SELinux 策略允许容器域访问该标签。

即使宿主用户可以 `ls` 目录，也不表示镜像中的应用 UID 一定能写。反过来，把目录改为 `777` 既不能解决所有 UID 映射问题，也会破坏最小权限。

⑤ [知识点] Rootless 端口限制是宿主策略，不应死记单一数字

非特权用户能否绑定某个低端口，取决于宿主机核参数和 Podman 网络实现等状态。调查入口：

```
sysctl net.ipv4.ip_unprivileged_port_start
```

在考试任务未要求低端口时，优先选择题目给出的高位宿主端口。端口权限问题不能靠切换为 rootful 容器掩盖，因为那会改变题目的安全和状态边界。

⑥ [边界] loginctl 在本章只用于观察，不用于配置持久运行

可观察目标用户的会话和 linger 状态：

```
loginctl show-user websvc -p State -p Linger
```

但 `loginctl enable-linger`、`systemctl --user` 与 Quadlet 属于下一章。当前容器 running 只证明当前实例，不证明注销或重启后的持续运行。

[Cheatsheet] rootless 的“root”仍是宿主普通用户；所有 Podman 命令保持目标用户一致；目录访问按宿主 DAC、UID/GID 映射和 SELinux 三层判断；低端口阈值查系统，不死记；持久运行留给下一章。

[操作专题] 组合名称、环境变量、端口发布与启动命令

`podman run` 的参数分成两部分：镜像名前是 Podman 的创建参数，镜像名后是交给容器执行的命令和参数。端口、环境和挂载在创建时固化；命令位置错误会导致 Podman 把参数解释给错误对象。

① [操作] 识别基本命令骨架

```
podman run [PODMAN OPTIONS] IMAGE [COMMAND [ARG...]]
```

示例：

```
podman run -d --name rh31-web \
registry.lab.example.com/training/webapp:1.0
```

若在镜像后追加命令，通常会覆盖或参与镜像的默认命令组合：

```
podman run --rm registry.lab.example.com/training/webapp:1.0 \
/bin/sh -c 'id && env | sort'
```

② [操作] 设置容器名称与环境变量

```
podman run -d --name rh31-web \
-e APP_MODE=exam \
-e LOG_LEVEL=info \
registry.lab.example.com/training/webapp:1.0
```

验证环境配置：

```
podman container inspect rh31-web
podman exec rh31-web /usr/bin/env
```

`exec` 只适用于 `running` 容器，而且容器中必须存在指定命令。`inspect` 是创建配置证据，`exec env` 是容器内部运行时证据，两者回答的问题不同。

`--user UID[:GID]` 改变容器内主进程使用的身份，不是把当前操作切换为另一个宿主用户。它必须结合镜像内账号、`rootless` `UID/GID` 映射和挂载目录权限判断。题目要求容器进程以特定身份运行时，应同时检查创建配置与容器内 `id`；不要把 `--user` 当作绕过宿主 `DAC` 或 `SELinux` 的办法。

③ [操作] 正确理解端口发布方向

```
-p 8088:8080
```

含义是：宿主端口 `8088` 转发到容器端口 `8080`。宿主端口在前，容器端口在后。

完整示例：

```
podman run -d --name rh31-web \
  -p 8088:8080 \
  registry.lab.example.com/training/webapp:1.0
```

不要根据镜像的 `EXPOSE` 元数据猜应用一定监听某端口。需要结合镜像说明、日志、容器内 `socket` 或任务参数确认容器端口。

④ [验证] 端口必须按三层证明

第一层：容器配置声明了什么？

```
podman port rh31-web
podman container inspect rh31-web
```

第二层：宿主是否出现相应监听？

```
ss -lnt
```

第三层：协议是否真的成功？

```
curl -v http://127.0.0.1:8088/
```

`podman port` 不能证明应用健康；`ss` 不能证明 `HTTP` 内容正确；`curl` 失败也不能直接证明端口发布错误，因为应用可能未监听、启动未完成或请求路径错误。

⑤ [诊断] Running 但不可访问

推荐链路：

```
podman ps
→ inspect 实际命令、环境和端口
→ logs 查看应用是否启动
→ podman port 查看映射
→ 必要时 exec/容器内 socket 检查
→ ss 查看宿主监听
→ curl 做协议验证
```

若发现把 `-p 8088:80` 写成了 `-p 80:8088`，端口配置不能通过 `stop/start` 修改；保存证据后重建容器。

[Cheatsheet] 镜像名前是 `Podman` 参数，镜像后是容器命令；`-p HOST:CONTAINER`；环境变量用 `inspect` 与容器内 `env` 双证据；端口按配置、监听、协议三层验收。

[知识专题] 容器数据到底放在哪里：可写层、Bind Mount 与 Named Volume

“容器里能看到文件”并不能回答数据是否持久。文件可能位于镜像只读层、容器可写层、宿主 `bind mount` 或 `Podman-managed volume`。删除和重建前必须先识别来源，否则最容易在容器可写层里丢失唯一数据。

① [知识点] 容器可写层与容器对象同生命周期

容器对镜像文件系统的普通修改通常进入该容器的可写层。停止并再次启动同一个容器时，这些数据仍可见；删除容器后，可写层随对象消失。

适合可写层的数据：临时缓存、无需跨重建保留的运行时文件。业务数据、配置输入和题目要求持久保存的文件不应只放在可写层。

② [知识点] Bind mount 把明确宿主路径暴露给容器

语法：

```
-v /HOST/PATH:/CONTAINER/PATH[:OPTIONS]
```

源路径由宿主管理，便于直接检查和备份。删除容器不会自动删除宿主源目录，但容器进程能否访问取决于宿主权限、UID/GID 映射和 SELinux 标签。

③ [知识点] Named volume 由 Podman 管理身份和路径

```
podman volume create webdata
podman volume inspect webdata
```

挂载：

```
-v webdata:/var/lib/webapp
```

显式命名 volume 独立于某个容器，便于多个重建实例复用。其实际挂载点由 Podman 管理，不应把内部路径当作稳定公共接口；检查和删除优先使用 `podman volume` 子命令。

④ [知识点] 匿名 Volume 与 Named Volume 的删除边界不同

省略 source：

```
-v /var/lib/webapp
```

Podman 会创建匿名 volume。匿名 volume 可以随 `podman run --rm` 或 `podman rm --volumes` 删除；显式命名 volume 不会被这些入口自动删除，需要单独管理。

⑤ [验证] 用 inspect 识别挂载来源，而不是只在容器内 ls

```
podman container inspect rh31-web
podman volume ls
podman volume inspect webdata
```

需要确认：

- Type 是 bind 还是 volume；
- Source 是哪个宿主路径或 volume；
- Destination 是容器内哪个路径；
- Options 是否包含只读、`z` 或 `Z`；
- 重建后是否仍指向同一数据来源。

[Cheatsheet] 可写层随容器删除；bind mount 明确指向宿主路径；named volume 独立管理；匿名卷可能随 `--rm/--volumes` 删除；持久性要通过 source 和重建验证。

[操作专题] 配置 Bind Mount、Named Volume 与 SELinux :Z/:z

挂载失败不是一个单一“权限问题”。稳定顺序是：先确认宿主路径和目标用户，再确认容器进程身份和 DAC，最后确认 SELinux。`chmod 777`、关闭 SELinux 或无条件 `--privileged` 都会掩盖真正的边界，并引入更大的风险。

① [操作] 准备 rootless 用户可管理的宿主目录

训练示例：

```
install -d -m 0750 -o webserv -g webserv /home/webserv/site
printf '%s\n' 'RHCSA-31 rootless web' > /home/webserv/site/index.html
chown webserv:webserv /home/webserv/site/index.html
```

随后切换到目标用户上下文：

```
sudo -iu webserv
id
```

实际考试中可能没有 `sudo` 或 `install`，可使用等价命令，但原则不变：源目录必须真实存在，目标用户必须可遍历和访问。

② [操作] 为单个容器使用私有重标记 `:Z`

```
podman run -d --name rh31-web \
-v /home/webserv/site:/opt/app/site:Z \
registry.lab.example.com/training/webapp:1.0
```

`:Z` 请求 Podman 为当前容器应用私有、非共享的 SELinux 标签。只有当前容器或具有匹配私有标签的对象应使用该内容。

③ [操作] 多个容器共享同一内容时使用 `:z`

```
-v /srv/shared-content:/opt/app/shared:z
```

`:z` 使用共享容器内容标签，适用于多个容器需要同时读写相同源目录的场景。不要仅因为记忆方便把所有挂载都写成 `:z`；共享边界应由任务决定。

④ [安全边界] 重标记会修改宿主文件标签

对大量 inode 递归重标记会延长容器启动时间；对系统目录、整个 `home` 或同时供其他受限服务使用的路径重标记，可能破坏其他服务。应优先为容器准备专用子目录，而不是把 `/home`、`/etc`、`/var` 等大范围路径直接加 `:Z`。

本章不把 `--security-opt label=disable` 作为常规答案。只有官方文档明确适用且任务安全边界允许时才考虑这种高级例外。

⑤ [操作] 创建并使用 Named Volume

```
podman volume create webdata
podman run -d --name rh31-web \
-v webdata:/var/lib/webapp \
registry.lab.example.com/training/webapp:1.0
```

验证：

```
podman volume ls
podman volume inspect webdata
podman container inspect rh31-web
```

⑥ [验证] 挂载要从配置、标签、容器和宿主四层证明

```
podman container inspect rh31-web
ls -ldZ /home/webserv/site
podman exec rh31-web ls -ldZ /opt/app/site
podman exec rh31-web sh -c 'test -r /opt/app/site/index.html'
cat /home/webserv/site/index.html
```

容器内创建测试文件后，还应在宿主源目录中确认同一文件，而不是只在容器内看见它。

[Cheatsheet] 源目录先存在且目标用户可访问；单容器私有用 `:Z`，多容器共享用 `:z`；重标记会修改宿主标签；不对系统大目录随意重标记；最终从 `inspect`、`ls -Z`、容器内访问和宿主文件四层验证。

[操作专题] 用 `ps`、`inspect`、`logs`、`exec` 与 `port` 建立运行证据

Podman 的多个查询命令并非同义词。`ps` 适合选择对象；`inspect` 读取结构化配置和状态；`logs` 读取日志驱动保存的 `stdout/stderr`；`exec` 在运行容器中执行定向命令；`port` 只列端口映射。排错时应让每条命令回答一个明确问题。

① [操作] 用 `podman ps` 和 `podman ps -a` 选择对象

```
podman ps
podman ps -a
podman ps -a --filter name=rh31-web
podman ps -a --filter status=exited
```

默认 `ps` 只显示 running 容器。容器立即退出、created 未启动或停止后，都必须用 `-a` 才能看到。

② [操作] 用显式对象类型的 `inspect` 避免歧义

```
podman image inspect IMAGE
podman container inspect CONTAINER
podman volume inspect VOLUME
```

虽然 `podman inspect` 能检查多种对象，讲义和考试答案中优先写清对象类型，便于确定自己正在读取镜像默认配置，还是容器实际创建配置。

容器 `inspect` 常见关注点：

- `.State.Status`；
- `.State.ExitCode`；
- `.Config.Cmd` 与镜像后的实际命令；
- 环境变量；
- 端口绑定；
- Mounts；
- 容器进程和创建时间。

③ [操作] 用 `logs` 读取 `stdout/stderr` 证据

```
podman logs rh31-web
podman logs --tail 50 rh31-web
podman logs --since 10m rh31-web
```

`podman logs` 只能显示容器日志驱动已经收集到的记录。应用只写内部文件、尚未刷新缓冲区或刚启动尚未输出时，空日志不能证明“没有错误”。

④ [操作] 用 `exec` 做最小、定向的容器内取证

```
podman exec rh31-web /usr/bin/env
podman exec rh31-web /bin/sh -c 'id; ls -ld /opt/app/site'
```

边界：

- 只能对 running 容器执行；
- 容器中必须存在相应命令；
- `exec` 成功只证明该命令在容器内成功，不证明宿主端口或外部业务成功；
- 不要把进入交互 Shell 当作唯一排错方法。

⑤ [操作] 用 `podman port`、`ss`、`curl` 完成外部链路

```
podman port rh31-web
ss -lnt
curl -v http://127.0.0.1:8088/
```

远程访问还会受到路由、`firewalld` 和上游网络设备影响，完整防火墙规则归第 21 章。本章只要求证明本机发布、监听和协议响应。

⑥ [帮助入口] 命令不确定时查本机版本

```
podman run --help
podman container inspect --help
man podman-run
man podman-inspect
man podman-volume
```

RHEL 9 不同小版本的 Podman 行为和字段可能不同，考试环境应以本机 man page 为最终命令边界。

[Cheatsheet] `ps` 选对象，`inspect` 看配置与状态，`logs` 看 stdout/stderr，`exec` 做容器内取证，`port/ss/curl` 看宿主链路；每条证据都有限定范围。

[诊断专题] 容器失败时按对象层推进，而不是先删除重建

一个稳定的容器诊断应从症状开始，把“镜像、容器对象、主进程、端口、挂载和宿主功能”逐层分开。只有下一条证据能区分假设时，才执行它；修复应尽量只改变已经证明错误的参数。

① [诊断] 容器立即退出

症状：`podman run -d` 返回 ID，但 `podman ps` 看不到。

当前证据：

```
podman ps -a --filter name=rh31-web
podman logs rh31-web
podman container inspect rh31-web
```

主要假设：

- 镜像默认命令本来就是一次性任务；
- 覆盖命令不存在或不可执行；
- 环境变量或配置缺失；
- 应用启动后主动报错退出；
- `bind mount` 覆盖了应用所需文件。

最小修复：根据退出码、实际命令和日志修正对应参数，不要用无限循环命令伪装“容器存活”。

再验证：`ps -a`、exit code、logs 和目标功能。

② [诊断] 容器 Running，但 `curl` 失败

当前证据：

```
podman ps --filter name=rh31-web
podman container inspect rh31-web
podman logs rh31-web
podman port rh31-web
ss -lnt
curl -v http://127.0.0.1:8088/
```

区分问题：

- 应用是否真正监听容器端口；
- 发布是否写成正确的 `HOST:CONTAINER`；
- 宿主是否监听预期地址；
- 应用是否尚未就绪；
- URL、路径或协议是否错误。

最小修复：端口配置错误时保留证据后重建；应用配置错误时修正环境或挂载；不要直接把问题归咎于 `firewalld`。

③ [诊断] Bind mount 报 `permission denied`

证据链：

```
目标用户能否遍历源目录
→ 源对象 owner/group/mode
→ 容器实际运行 UID/GID 与映射
→ inspect 的 Mounts 和 options
→ ls -Z / AVC 证据
→ 只修复错误层
```

推荐命令：

```
namei -l /home/websvc/site
ls -ldZ /home/websvc/site
podman container inspect rh31-web
podman unshare id
```

需要 SELinux AVC 时，引用第 28、29 章的方法，不在本章完整展开。默认修复不是 `chmod 777`、`setenforce 0` 或 `--privileged`。

④ [诊断] Root 看不到普通用户的容器

症状：管理员执行 `podman ps -a` 为空，但用户声称容器存在。

最有区分度的证据：

```
sudo -iu websvc podman ps -a
sudo podman ps -a
```

若前者存在而后者为空，这是正常的每用户状态隔离，不是容器丢失。所有后续 `inspect`、`logs`、`stop` 和 `rm` 必须回到 `websvc` 上下文。

⑤ [诊断] 删除容器后数据消失

先回答数据原来在哪里：

```
podman container inspect OLD_CONTAINER
```

若没有 `bind mount` 或 `named volume`，文件很可能只在可写层。已经删除且没有备份时，本章不能承诺恢复。后续应把业务数据迁移到明确的宿主目录或 `named volume`，并通过删除重建测试证明持久性。

⑥ [安全边界] 何时拒绝“快捷修复”

以下做法不能作为默认答案：

- `podman rm -f`：会破坏现场和有序停止；
- `--privileged`：扩大容器对宿主的访问能力；
- `chmod 777`：破坏最小权限且不能解决 SELinux；
- 关闭 SELinux：绕过策略而非修复标签和访问模型；
- 改用 `rootful` 容器：改变题目要求和状态所有者；
- 只看 `running`：不能证明端口、协议和数据。

[Cheatsheet] 立即退出查 `ps -a/logs/inspect`；不可访问查应用、发布、监听、协议；挂载拒绝按 DAC、映射、SELinux 分层；`root` 与普通用户状态分开；删除前先确认数据来源。

[经典任务] 部署带端口、环境变量和持久目录的 Rootless Web 容器

以下参数用于训练，不代表真实考试基础设施。

环境

- 主机已安装 Podman；
- 普通用户 `websvc` 已存在，并具备 rootless Podman 所需 subordinate UID/GID；
- 镜像为 `registry.lab.example.com/training/webapp:1.0`；
- 容器内应用监听 TCP `8080`；
- 宿主目录 `/home/websvc/site` 已包含必须保留的 `index.html`；
- SELinux 保持 Enforcing。

当前状态

- 可能存在名为 `rh31-web` 的 `stopped` 容器；
- 不确定目标用户是否已经拉取镜像；
- 不允许删除宿主目录中的现有文件。

目标终态

1. 全部 Podman 操作以 `websvc` 身份执行；

2. 当前用户本地存储中存在指定完整镜像；
3. 容器名为 `rh31-web`；
4. 设置环境变量 `APP_MODE=exam`；
5. 发布宿主 `8088` 到容器 `8080`；
6. bind mount `/home/websvc/site` 到 `/opt/app/site`，并为单容器选择正确 SELinux 重标记；
7. 容器当前为 running；
8. 宿主 `curl http://127.0.0.1:8088/` 能读取现有页面；
9. 容器删除并按同样配置重建后，宿主目录数据仍存在。

限制条件

- 不使用 `--privileged`；
- 不关闭 SELinux；
- 不执行 `chmod 777`；
- 不使用 `root` 创建同名容器；
- 不无调查执行 `podman rm -f`；
- 不配置 `user systemd`、`linger` 或 `Quadlet`。

验收证据

```
id / podman info
→ image inspect
→ ps -a / container inspect
→ logs
→ podman port
→ ss
→ curl
→ 宿主文件
→ 删除重建后的数据
```

请先独立完成。参考解答从下一页开始。

[参考解答] 任务一：从用户身份到数据重建的分层实施

本解答只提供推荐命令和判断逻辑，未在本会话控制的 RHEL 9 虚拟机中执行。实际考试应按题目给出的主机名、镜像、端口和路径替换训练参数。

① [调查] 进入目标用户上下文并建立基线

管理员先确认账号和映射：

```
id websvc
grep '^websvc:' /etc/subuid /etc/subgid
```

进入目标用户：

```
sudo -iu websvc
id
podman info
podman ps -a --filter name=rh31-web
podman images
```

若同名容器存在，先检查：

```
podman container inspect rh31-web
podman logs rh31-web
```

- 若对象配置完全正确，只是 stopped，可考虑 `podman start rh31-web`；
- 若端口、环境或挂载错误，保存证据后 `stop/rm` 重建；
- 不直接 `rm -f`。

② [调查] 核对宿主目录

```
ls -ldZ /home/websvc/site
ls -lZ /home/websvc/site
cat /home/websvc/site/index.html
```

确认 `websvc` 可遍历和读取。若任务要求容器写入，还要确保映射后的应用 UID/GID 具备所需写权限；不能仅用 `777` 解决。

③ [操作] 拉取并检查镜像

```
podman pull registry.lab.example.com/training/webapp:1.0
podman images
podman image inspect registry.lab.example.com/training/webapp:1.0
```

检查镜像实际存在于 `websvc` 的本地存储，并确认默认命令、架构和镜像身份。

④ [操作] 删除已证明配置错误的旧对象

仅当基线证明必须重建：

```
podman stop rh31-web 2>/dev/null || true
podman rm rh31-web
```

若容器已经 `stopped`，`stop` 可能不需要。命令中的 `|| true` 只是说明流程容忍“对象本来未运行”，考试中也可以先判断状态后执行更精确命令。

⑤ [操作] 创建正确容器

```
podman run -d --name rh31-web \
-e APP_MODE=exam \
-p 8088:8080 \
-v /home/websvc/site:/opt/app/site:Z \
registry.lab.example.com/training/webapp:1.0
```

参数解释：

- `-d`：后台运行，不保证应用一定持续运行；
- `--name rh31-web`：建立稳定管理名称；
- `-e APP_MODE=exam`：写入创建配置；
- `-p 8088:8080`：宿主 8088 到容器 8080；
- `:Z`：单容器私有重标记；
- 最后是完整镜像引用。

⑥ [验证] 容器对象和主进程

```
podman ps -a --filter name=rh31-web
podman container inspect rh31-web
podman logs --tail 50 rh31-web
```

必须确认：

- 状态是 `running`；
- 实际环境包含 `APP_MODE=exam`；
- Mounts 指向正确 `source/destination`；
- 端口绑定方向正确；
- 日志中没有立即退出或配置错误。

⑦ [验证] 宿主监听和 HTTP

```
podman port rh31-web
ss -lnt
curl -v http://127.0.0.1:8088/
```

`curl` 返回目标页面才证明本机 HTTP 功能；只有 `podman ps` 为 `running` 不够。

⑧ [验证] 数据确实来自宿主目录

```
cat /home/websvc/site/index.html
podman exec rh31-web cat /opt/app/site/index.html
```

两处内容应一致。若容器写入测试文件：

```
podman exec rh31-web sh -c \
'printf "%s\n" container-write > /opt/app/site/container-write.txt'
cat /home/websvc/site/container-write.txt
```

只有宿主源目录能看到同一文件，才证明写入进入 bind mount，而不是其他路径。

⑨ [验证] 删除并重建后数据仍在

先保存创建参数或 inspect 证据，再：

```
podman stop rh31-web
podman rm rh31-web
cat /home/websvc/site/index.html
```

数据仍在宿主目录。按相同 `podman run` 命令重建后：

```
podman run -d --name rh31-web \
-e APP_MODE=exam \
-p 8088:8080 \
-v /home/websvc/site:/opt/app/site:Z \
registry.lab.example.com/training/webapp:1.0

curl http://127.0.0.1:8088/
```

这一步证明数据持久性来自 bind mount，而不是旧容器可写层。

⑩ [边界] 本任务未证明注销或重启后自动运行

可观察：

```
loginctl show-user websvc -p State -p Linger
```

但本章不执行 `enable-linger`，也不创建 user systemd 或 Quadlet。当前验收仅到“当前 rootless 容器、端口和数据正确”。持久运行归下一章。

[Cheatsheet] 目标用户基线 → 目录 → 镜像 → 必要时证据重建 → `run(name/env/port/volume)` → `ps/logs/inspect` → `port/ss/curl` → 宿主数据 → 删除重建后再验。

[经典任务] 修复 Running 但不可访问且挂载被 SELinux 拒绝的容器

环境与症状

普通用户 `websvc` 已运行名为 `rh31-broken` 的 rootless 容器。镜像内应用监听 `8080`，但现有容器错误地发布为：

```
8088:80
```

同时把 `/home/websvc/site` 挂载到 `/opt/app/site`，创建时没有使用 `:Z` 或 `:z`。当前证据：

- `podman ps` 显示 running；
- `curl http://127.0.0.1:8088/` 失败；
- 应用日志包含访问站点目录失败的信息；
- SELinux 保持 Enforcing。

目标

1. 保留并读取现有容器证据；
2. 证明端口发布方向和容器应用端口不匹配；
3. 按 DAC、UID/GID 映射与 SELinux 标签区分挂载失败层；

4. 不关闭 SELinux、不使用 `--privileged`、不执行 `chmod 777`；
5. 以最小参数修正后重建；
6. 重新证明 running、正确端口、宿主监听、HTTP 和宿主数据。

[参考解答] 任务二：把两个故障拆成端口层与挂载层

① [调查] 保存当前容器配置、状态与日志

```
podman ps -a --filter name=rh31-broken
podman container inspect rh31-broken > rh31-broken.inspect.json
podman logs rh31-broken > rh31-broken.log 2>&1
podman port rh31-broken
```

从 inspect 和 port 证明宿主 `8088` 实际映射到容器 `80`，而应用要求容器 `8080`。此时已经证明必须重建，单纯 restart 无法修改发布配置。

② [调查] 检查挂载和宿主 DAC

```
namei -l /home/websvc/site
ls -ldZ /home/websvc/site
podman container inspect rh31-broken
```

确认 source/destination 正确，`websvc` 能遍历源路径，且挂载 options 中缺少正确 SELinux 重标记。若 DAC 本身错误，先按最小权限修正 owner/group/mode；若 DAC 已满足，再进入 SELinux 证据。

③ [调查] 读取 SELinux 证据

本章只给出入口，完整 AVC 解释归 SELinux 章节：

```
ls -ldZ /home/websvc/site
sudo ausearch -m AVC -ts recent
```

如果证据指向容器域被宿主目录标签拒绝，使用专用目录和 `:Z` 是比关闭 SELinux 更小的修复。

④ [操作] 有序停止并删除错误对象

```
podman stop rh31-broken
podman rm rh31-broken
```

旧 inspect 和 logs 已保存，可用于对比。

⑤ [操作] 按正确端口和私有挂载重建

```
podman run -d --name rh31-broken \
  -p 8088:8080 \
  -v /home/websvc/site:/opt/app/site:Z \
  registry.lab.example.com/training/webapp:1.0
```

若多个容器确实需要共享该目录，应根据共享边界改用 `:z`；不能仅因 `:z` 更宽松就默认使用。

⑥ [分层验证]

```
podman ps -a --filter name=rh31-broken
podman logs --tail 50 rh31-broken
podman container inspect rh31-broken
podman port rh31-broken
ss -lnt
curl -v http://127.0.0.1:8088/
ls -ldZ /home/websvc/site
podman exec rh31-broken test -r /opt/app/site/index.html
cat /home/websvc/site/index.html
```

验收结论必须逐层表达：

- `ps -a`：当前主进程是否运行：

- inspect: 创建配置是否已修正；
- logs: 应用是否仍报错；
- port/ss: 发布和宿主监听；
- curl: HTTP 功能；
- `ls -Z` 与容器读取：挂载和 SELinux；
- 宿主文件：数据来源。

[Cheatsheet] 先保存旧 inspect/logs → 证明 `8088:80` 错误 → 检查 DAC/映射/标签 → stop/rm → `8088:8080` + `:Z` 重建 → 对象、日志、端口、监听、HTTP、数据逐层再验。

[本章收束] 把“容器能跑”还原成一条可证明的状态链

本章最重要的工作方法，是始终先问“当前证据属于哪一个对象层”。镜像、容器对象、容器主进程、宿主监听、协议功能和外部数据不是同一个状态；rootless 又要求所有操作和查询保持在同一宿主用户上下文。只有把这些层次分开，才不会把局部成功扩大为最终完成。

工作方法：从身份到功能逐层推进

- 确认目标宿主用户
- 证明镜像引用与本地镜像身份
 - 记录容器创建配置
 - 判断主进程状态与退出码
 - 检查端口发布、挂载和环境
 - 证明宿主监听与协议功能
 - 证明数据位于可持久来源
 - 必要时保存证据后最小重建

每次验证都要同时写清“能证明什么”和“不能证明什么”。`podman ps` 中的 `running` 只证明主进程仍存在；`podman port` 只证明 Podman 记录了映射；`ss` 只证明宿主 socket；`curl` 才能继续证明 HTTP 请求；宿主源目录中的文件则证明数据真正进入外部存储。

主要判断表

看到的证据	可以证明	仍不能证明
<code>podman images</code> 中存在目标引用	当前用户本地存储存在该引用	容器已经创建或正在使用它
<code>podman ps</code> 显示 <code>running</code>	容器主进程当前仍运行	应用已就绪、端口可达、数据正确
<code>podman port NAME</code> 有映射	创建配置中存在端口发布	宿主 socket 和协议功能成功
<code>ss -lnt</code> 出现宿主端口	本机存在对应 TCP 监听	HTTP 内容或远端访问正确
<code>podman inspect</code> 显示挂载	容器配置引用了某个 source	进程拥有 DAC/映射/SELinux 访问权
<code>curl</code> 返回目标内容	当前请求链在本机成功	注销或重启后仍会自动运行
删除容器后宿主文件仍在	数据不只位于旧容器可写层	下一次创建参数一定正确

向下一章交接

本章的终点是“当前 rootless 容器实例、端口、环境、挂载和数据可验证”。用户退出后容器是否继续、主机启动后如何自动创建或启动、如何把 `podman run` 参数转化为可维护的 Quadlet 声明，以及怎样验证冷启动，全部进入下一章《容器持久化、用户 systemd 与 Quadlet》。本章不执行 `loginctl enable--linger`，也不把当前 `running` 当作持久运行证据。

章末速查

```
# 用户与存储上下文
id
podman info
podman images

# 容器对象与运行证据
podman ps -a
podman container inspect NAME
podman logs --tail 50 NAME
podman port NAME

# 宿主功能与数据
ss -lnt
curl -fsS http://127.0.0.1:HOST_PORT/
ls -ldZ HOST_PATH
podman exec NAME id
podman exec NAME test -r CONTAINER_PATH

# 删除前的数据边界
podman inspect NAME --format '{{json .Mounts}}'
podman volume ls
podman volume inspect VOLUME
```

静态验证声明：本章依据 RHEL 9 课程、Red Hat 容器文档、Podman 手册、细分课件和模拟题进行静态核对。当前未执行命令级 live test；涉及版本默认值、用户映射、网络后端和 SELinux 标签变化的结论，应在目标 RHEL 9 环境中再次验证。